

CSE301 ALGORITHMS
ALGORITHMS CONTEST ASSIGNMENT



Biweekly Contest #172

AKDENİZ UNIVERSITY

DEPARTMENT OF COMPUTER ENGINEERING

FALL 2025-2026 SEMESTER

YAHYA EFE KURUÇAY
20220808005

BURAK YALÇIN
20220808069

20 December 2025, 5.30 pm GMT+3

TABLE OF CONTENTS

1. RANKING PAGE

2. PROBLEM AND SOLUTION PAGES

- 2.1 PROBLEM 1 AND SOLUTION
 - 2.1.1 ACCEPTED SCREEN 1
- 2.2 PROBLEM 2 AND SOLUTION
 - 2.2.1 ACCEPTED SCREEN 2
- 2.3 PROBLEM 3 AND SOLUTION
 - 2.3.1 ACCEPTED SCREEN 3
- 2.4 PROBLEM 4 AND SOLUTION
 - 2.4.1 ACCEPTED SCREEN 4

1. RANKING PAGE

Ranking of Biweekly Contest 172

Global LLM 30344 2306 AK1

Ended

Rank	Name	Score	Finish Time	Q1 (4)	Q2 (4)	Q3 (5)	Q4 (6)
1595	You	19	01:27:18	00:47:32	01:05:59	01:09:47	01:12:18
1	Divyansh	19	00:04:33	00:02:25	00:03:10	00:03:52	00:04:33
2	pengFarmer	19	00:05:25	00:03:17	00:02:47	00:03:49	00:05:25
3	Lily_milk	19	00:07:00	00:05:00	00:04:17	00:07:00	00:05:42
4	Arushi-Ojha	19	00:08:38	00:04:20	00:05:50	00:07:38	00:08:38
5	Hachiko	19	00:09:08	00:02:46	00:04:12	00:06:08	00:09:08
6	raj_soni	19	00:10:09	00:02:23	00:10:09	00:07:46	00:05:03
7	kzzfady156	19	00:10:18	00:03:16	00:07:45	00:09:11	00:10:18
8	PytsBestLang	19	00:10:47	00:02:05	00:04:00	00:06:52	00:10:47
9	TkTVI	19	00:11:49	00:01:43	00:06:41	00:07:46	00:11:49
10	流蘇之韻	19	00:11:56	00:01:42	00:04:17	00:07:41	00:11:56
11	XYZzzz_20	19	00:12:24	00:03:37	00:04:55	00:06:25	00:07:24
12	jam	19	00:12:33	00:03:37	00:06:10	00:08:49	00:12:33
13	__baozh__	19	00:12:50	00:01:11	00:03:38	00:05:47	00:12:50

2. PROBLEM AND SOLUTION PAGES

2.1 PROBLEM 1 AND SOLUTION

The screenshot shows the LeetCode interface for the problem "Q1. Minimum Number of Operations to Have Distinct Elements". The problem is marked as "Solved" and "Medium" (4 pts). The description states: "You are given an integer array `nums`. In one operation, you remove the first three elements of the current array, if there are fewer than three elements remaining, all remaining elements are removed. Repeat this operation until the array is empty or contains no duplicate values. Return an integer denoting the number of operations required."

Examples:

- Example 1: Input: `nums = [3,8,3,6,5,8]`, Output: 1. Explanation: In the first operation, we remove the first three elements. The remaining elements `[6, 5, 8]` are all distinct, so we stop. Only one operation is needed.
- Example 2: Input: `nums = [2,2]`, Output: 1. Explanation: After one operation, the array becomes empty, which meets the stopping condition.
- Example 3: Input: `nums = [4,3,5,1,2]`, Output: 0. Explanation: The array is already empty.

The code editor on the right shows a Java solution:

```
1 class Solution {
2     public int minOperations(int[] nums) {
3         int n = nums.length;
4
5         Set<Integer> seen = new HashSet<>();
6         int bestStart = 0;
7
8         for (int i = n - 1; i >= 0; i--) {
9             if (seen.contains(nums[i])) {
10                 bestStart = i + 1;
11                 break;
12             }
13             seen.add(nums[i]);
14         }
15
16         int toRemove = bestStart;
17         return (toRemove + 2) / 3;
18     }
19 }
```

2.1.1 ACCEPTED SCREEN 1

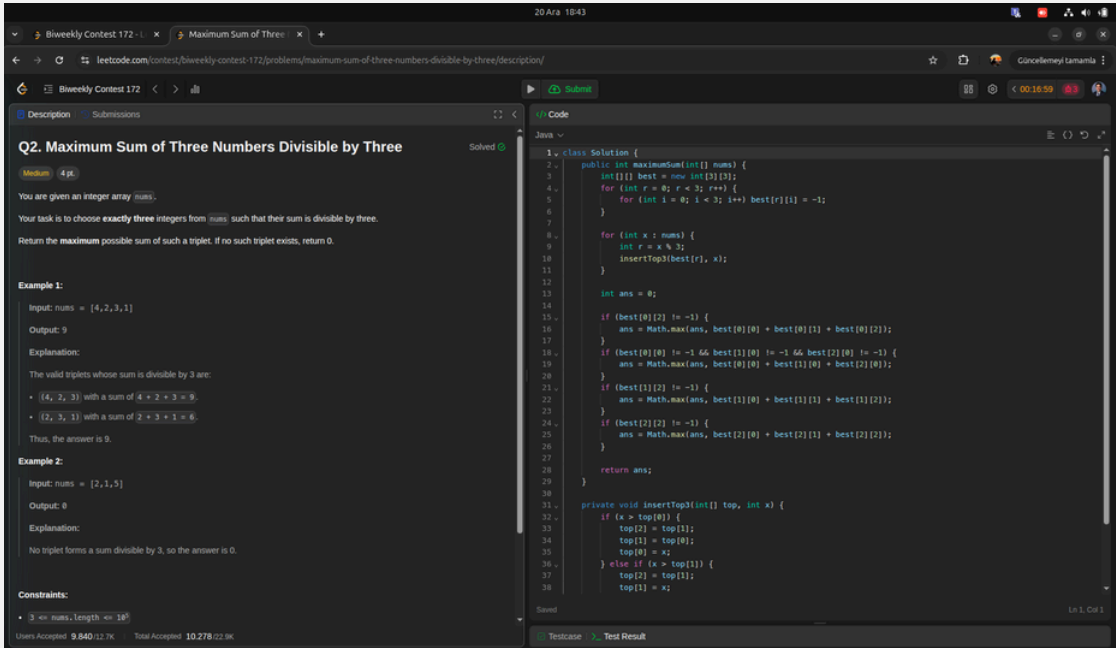
The screenshot shows the submission page for the problem. It indicates "Accepted 728 / 728 testcases passed" and "UOKhkaNoI2 submitted at Dec 20, 2025 18:34". The performance metrics are:

- Runtime: 10 ms
- Memory: 139.89 MB

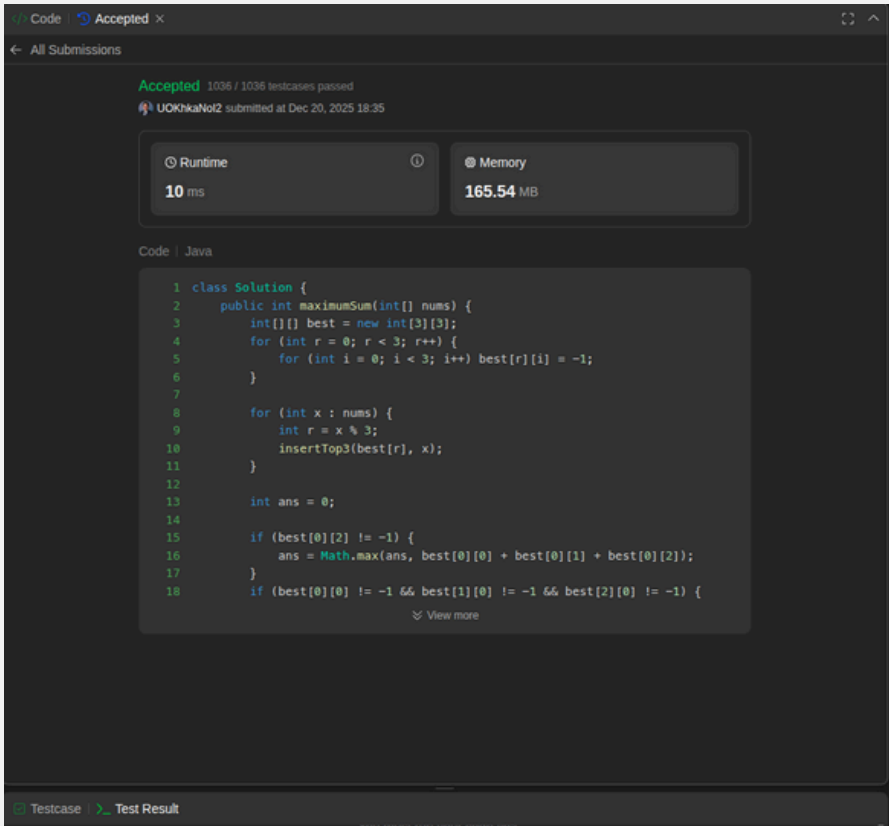
The code editor shows the same Java solution as in the previous screenshot:

```
1 class Solution {
2     public int minOperations(int[] nums) {
3         int n = nums.length;
4
5         Set<Integer> seen = new HashSet<>();
6         int bestStart = 0;
7
8         for (int i = n - 1; i >= 0; i--) {
9             if (seen.contains(nums[i])) {
10                 bestStart = i + 1;
11                 break;
12             }
13             seen.add(nums[i]);
14         }
15
16         int toRemove = bestStart;
17         return (toRemove + 2) / 3;
18     }
19 }
```

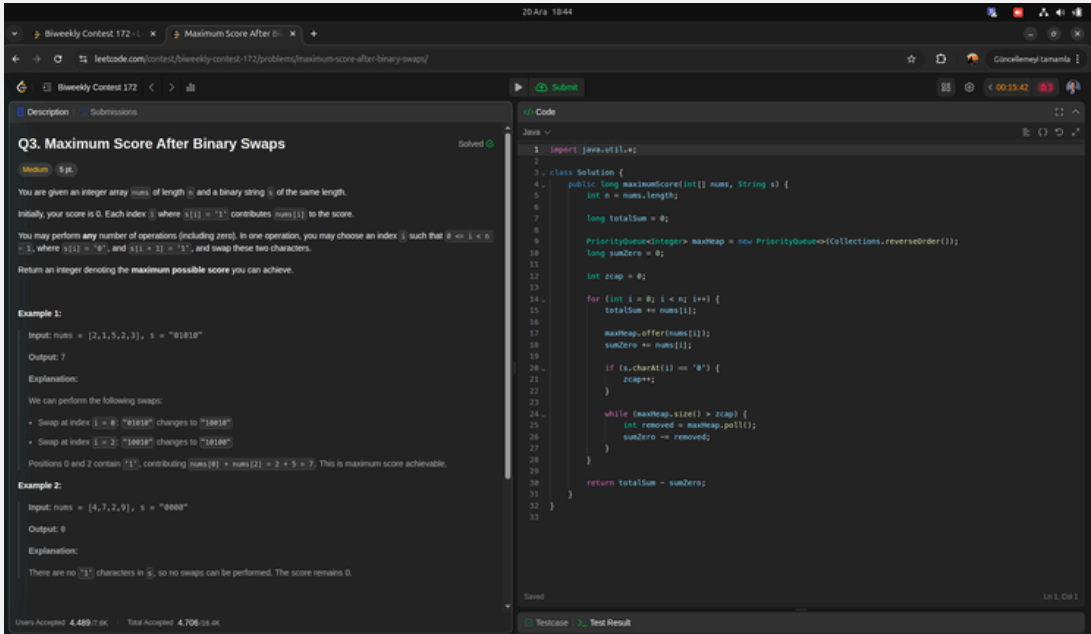
2.2 PROBLEM 2 AND SOLUTION



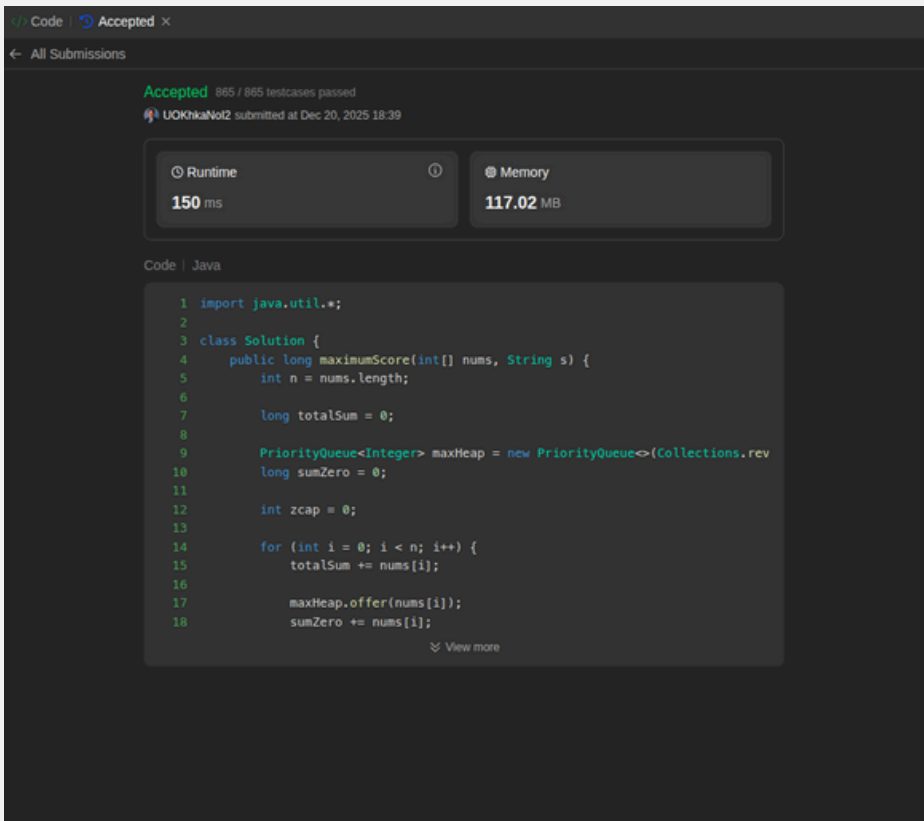
2.2.1 ACCEPTED SCREEN 2



2.3 PROBLEM 3 AND SOLUTION



2.3.1 ACCEPTED SCREEN 3



2.4 PROBLEM 4 AND SOLUTION

The screenshot shows the LeetCode interface for the problem "Q4. Last Remaining Integer After Alternating Deletion Operations". The problem description states: "You are given an integer n. We write the integers from 1 to n in a sequence from left to right. Then, alternately apply the following two operations until only one integer remains, starting with operation 1: Operation 1: Starting from the left, delete every second number. Operation 2: Starting from the right, delete every second number. Return the last remaining integer." Example 1 shows n=8, resulting in 3. Example 2 shows n=5, resulting in 1. The Java solution is as follows:

```
1 class Solution {
2     public long lastInteger(long n) {
3         long start = 1;
4         long step = 1;
5         long len = n;
6
7         boolean oLeft = true;
8
9         while (len > 1) {
10             if (oLeft) {
11                 if ((len & 1) == 0) {
12                     start += step;
13                     start += step;
14                 }
15             }
16             step <<= 1;
17             len = (len + 1) >> 1;
18             oLeft = !oLeft;
19         }
20         return start;
21     }
22 }
```

The test result shows "Finished" with a runtime of 0 ms and memory usage of 43.13 MB.

2.4.1 ACCEPTED SCREEN 4

The screenshot shows the "Accepted" submission page for the problem "Last Remaining Integer After Alternating Deletion Operations". The submission was made by "UOKhkaNoI2" on Dec 20, 2025, at 18:51. The submission passed 1310 / 1310 testcases. The runtime is 1 ms and the memory usage is 43.13 MB. The code is in Java and is as follows:

```
1 class Solution {
2     public long lastInteger(long n) {
3         long start = 1;
4         long step = 1;
5         long len = n;
6
7         boolean oLeft = true;
8
9         while (len > 1) {
10             if (oLeft) {
11                 if ((len & 1) == 0) {
12                     start += step;
13                     start += step;
14                 }
15             }
16             step <<= 1;
17             len = (len + 1) >> 1;
18             oLeft = !oLeft;
19         }
20         return start;
21     }
22 }
```